

CS 224R Milestone Report: Self-Improving Agent Harnesses via Trace-Derived RL Environments for Custom-Kernel Engineering

Chandra Suda Aaditya Nalawade | Stanford | May 23, 2026

1. Experiments so far

For the milestone we narrowed the original objective (self-improving agent harnesses via trace-derived RL environments) to its first concrete verifier domain, GPU kernel engineering (§2), and built kernel-synth (<https://github.com/chandrasuda/kernel-synth>): a trace-derived environment factory that shallow-clones a GitHub repo, walks its AST to enumerate every `nn.Module` subclass, and selects a diverse, non-obvious subset using either our custom tool-use agent harness or a deterministic AST heuristic favoring modules with unusual forward operations or imports of known custom-kernel libraries (Triton, FlashAttention, Mamba SSM). Each selected module is converted into a self-contained RL environment: a frozen reference, an auto-inferred input generator, an editable solution scaffold, a benchmarking harness, and metadata; a local FastAPI viewer browses the catalog. On top of this we wired a Harbor-style ATIF v1.7 rollout layer that uses `torch.compile` as the reward ceiling and exposes Triton-only agent tools inside a workspace sandbox; baseline and `torch.compile` rollouts already validate against the schema.

1.1 Required experiment

We ran the pipeline end-to-end in deterministic-heuristic mode on five PyTorch repos chosen for mechanism diversity:

repo	py files	kept	avg novelty	top picks
state-spaces/mamba	55	8	0.83	Mamba2, Mamba3, MHA
lucidrains/vit-pytorch	76	8	0.92	DSSA, NaViT, LookViT
lucidrains/x-transformers	27	8	0.96	CoPE, DataDependentAlibi, HyperConnection
openai/whisper	14	3	0.21	(down-weighted: vanilla transformer)
lucidrains/audiolm-pytorch	14	8	0.75	Attend, SoundStream, HubertWithKmeans
total	186	35	0.81	~60k LOC scanned

1.2 Did results match expectations?

Largely yes. We expected the heuristic to down-weight vanilla transformer code and up-weight unusual mechanisms; it did exactly that (Whisper $\Rightarrow$ avg 0.21; CoPE, DataDependentAlibi, Mamba2/3, DSSA $\Rightarrow$ near 1.0). This is the right inductive bias for our reward, since we don't want environments on modules where a fused kernel already exists (the speedup ceiling collapses). Surprises. The first converter naïvely copied the source file's full import block, breaking many environments by pulling in unrelated dependencies. The fix (walk the class body for referenced names, keep only the imports actually used, transitively lift any helper definitions the class references) was straightforward but required a build-time correctness probe we hadn't planned for: trace-derived environments are only useful if the reference module actually runs.

2. Changes to hypothesis / objective

The closed-loop hypothesis is unchanged: agents improve faster when their RL training environments are mined from real trace/code distributions than when they are hand-designed. What narrowed is the verifier ("general ML-research harness defects" $\rightarrow$ "GPU kernel engineering"), because kernels admit a cheap, fully-automatable, dense reward (numerical equivalence + wall-clock speedup vs `torch.compile`), which is exactly what makes long-horizon RL tractable; general ML-research decisions are subjective, slow to verify, and contaminated by API/cost noise, so they are better attacked once the loop is proven on the kernel sub-domain. Preserved: trace-derived envs, GRPO+LoRA training, evaluation vs base / SFT / prompt-only baselines, ablations over env-derivation vs synthetic. New: a concrete environment factory (kernel-synth), an external expert baseline (`torch.compile`) defining the reward ceiling, and Triton as a much smaller action space than the full agentic one (substantially easier to RL).

3. Steps to completion

(1) Harden the rollout layer: stress-test the benchmark pipeline, tighten the agent-mode loop, improve auto-inferred input generators so more environments run end-to-end out of the box. (2) Generate trajectories at scale: latest frontier and open-source coding LLMs over 35 + 5 held-out environments, $N \approx 200$ ATIF trajectories with rewards in $\{-0.1\} \cup [-0.2, 1.5]$. (3) GRPO+LoRA on Qwen3-1.7B or SmolLM3-3B with $r = \text{clip}((t_e - t_s)/(t_e - t_c), -0.2, 1.5)$ (t_e, t_s, t_c = eager/solution/compile latency) gated by correctness; evaluated vs base, prompt-only self-reflection, and SFT. (4) Ablations: heuristic vs agent env-selection; trace-derived vs generic Triton-toy curriculum; `torch.compile` ceiling on/off. (5) Final eval: best-of- N solve rate, mean reward, fraction reaching $r \geq 0.5$, and t_s/t_c distribution.

3.1 Realism and risks

Remaining work is trajectory generation + a small RL loop on a $\leq 3\text{B}$ model, both already prototyped. Risks: (i) CPU `torch.compile` on small modules can be slower than eager, compressing the reward gap (mitigation: CUDA-aware benchmark presets); (ii) several environments still need hand-tuned input shapes (mitigation: auto-detection of Embedding/Conv consumers); (iii) environment distribution may concentrate around attention variants (mitigation: held-out repos from disjoint mechanism families).

4. Team contributions

Chandra Suda: kernel-synth pipeline, AST heuristic, environment converter, FastAPI viewer, ATIF rollout layer; will own agent-mode trajectories, held-out eval, env-selection ablations. Aaditya Nalawade: related-work survey, reward-shape design, GRPO+LoRA scaffold (Qwen3-1.7B), KernelBench-mini baseline; will own training-loop integration, failure-mode analysis. Shared: error analysis, write-up, presentation.